

65. Chercher dans un tableau

Sortir dès qu'on a trouvé, et réserver une valeur pour « pas trouvé ».

1. Ce qu'il faut comprendre

`break` sort de la boucle ; `continue` passerait au tour suivant.

`ou = 255` sert de « pas trouvé ». Sur un octet il n'y a pas de valeur spéciale, alors **on en réserve une** — et l'on choisit celle qui ne peut pas être un index valide, puisqu'un tableau fait 256 cases au plus.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **627 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
main()	109 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← <code>EcrireTexte</code> , <code>EcrireNombre</code> – attend de pouvoir écrire une case
EcrireTexte	10 o	← <code>texte()</code> – écrit un texte, lettre par lettre
EcrireNombre	33 o	← <code>nombre()</code> – écrit un nombre en chiffres
CopierTuiles	67 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	272 o	← <code>CopierTuiles</code> – 17 lettres sur 43
"TROUVE EN"	9 o	← <code>texte()</code> – un texte gravé

Lettres gravées : ENORTUV0123456789

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>texte()</code>	× 1	11 o sur place, dans <code>main()</code>	+ <code>EcrireTexte</code> (10 o)
<code>nombre()</code>	× 1	10 o sur place, dans <code>main()</code>	+ <code>EcrireNombre</code> (33 o)
<code>image()</code>	× 1	3 o sur place, dans <code>main()</code>	+ <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : remplit un tableau de 8 nombres (0, 3, 6, 9 ...), y cherche
// le nombre 12, et affiche à quelle case il l'a trouvé.
//
// break : sort de la boucle TOUT DE SUITE (inutile de continuer à chercher).

#include <texte>      // écrit un texte à l'écran
#include <nombre>    // écrit un nombre en chiffres
```

2. UN TABLEAU — VALEURS

```
uint8_t valeurs[8];           // 8 cases : valeurs[0] à valeurs[7].
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                  // Le jeu commence ici.
    for (uint8_t i = 0; i < 8; i++) valeurs[i] = i * 3;
                                // 0, 3, 6, 9, 12, 15, 18, 21

    uint8_t cherche = 12;      // Le nombre qu'on cherche.
    uint8_t ou = 255;          // Où on l'a trouvé. 255 = « pas trouvé » (une valeur impossible).

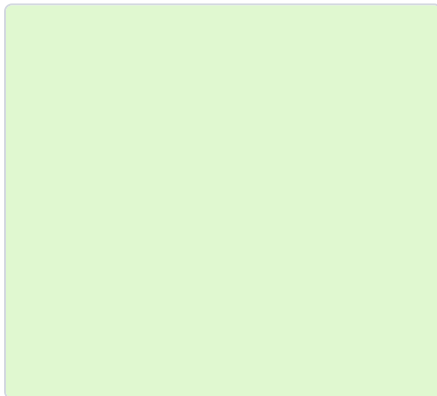
    for (uint8_t i = 0; i < 8; i++) {                // Chaque case i :
        if (valeurs[i] == cherche) { ou = i; break; } //   c'est lui ? on note i, et on sort.
    }                                                  // valeurs[4] vaut 12 → ou = 4.

    texte(2, 6, "TROUVE EN");
    nombre(13, 6, ou);           // Affiche 004.

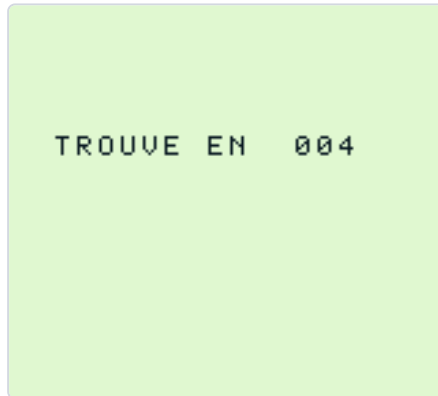
    while (true) {               // Répète pour toujours :
        image();                 //   attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« TROUVE EN 004 » — 12 est bien la cinquième valeur.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la recherche a abouti (4)

- ✓ ce n'est pas resté à « pas trouvé »
- ✓ et le résultat est affiché
- ✓ le tableau contient bien des multiples de trois

Ensuite : **66. Un texte qui porte un nom** — chapitre 9 : Ranger ce qu'on manipule. · gameboy3 — écrire une cartouche Game Boy en C++